

One effective technique for addressing class imbalance in machine learning is oversampling, wherein the minority class is oversampled by duplicating its examples in the training dataset before fitting the model. While this rebalances the class distribution, it doesn't offer any new information to the model. Upsampling can enhance model performance, particularly in identifying rare events or anomalies. It is worth noting that the methods explained below are not exhaustive, but offer practical insights.

Resample

- Resample upsamples a dataset by simply duplicating records from minority classes.
- *Logic*
 - The default strategy involves executing the bootstrapping procedure.
 - Bootstrapping entails creating subsets from a dataset multiple times.
 - Depending on the size chosen, it basically creates a random selection with or without replacement of the dataset.
 - *Sampling without replacement:* A subset of observations is randomly selected, and once an observation is chosen, it cannot be selected again.
 - *Sampling with replacement:* A subset of observations is randomly selected, and an observation may be chosen multiple times.
 - The resample function is specifically used to resample a dataset *n_samples* times, with the default option being to sample with replacement.
- Sample code is given below:

```
from collections import Counter
from sklearn.datasets import make_classification
from sklearn.utils import resample
from matplotlib import pyplot
from numpy import where
import pandas as pd
import numpy as np

X, y = make_classification(n_classes=2, class_sep=2,
weights=[0.1, 0.9], n_features=4, n_clusters_per_class=1,
n_samples=1000, random_state=10)

# summarize class distribution
print('Original dataset shape %s' % Counter(y))
Original dataset shape Counter({1: 894, 0: 106})

# scatter plot of examples by class label
for label, _ in Counter(y).items():
    row_ix = where(y == label)[0]
    pyplot.scatter(X[row_ix, 0], X[row_ix, 1], label=str(label))
```

```
pyplot.legend()
pyplot.show()
```

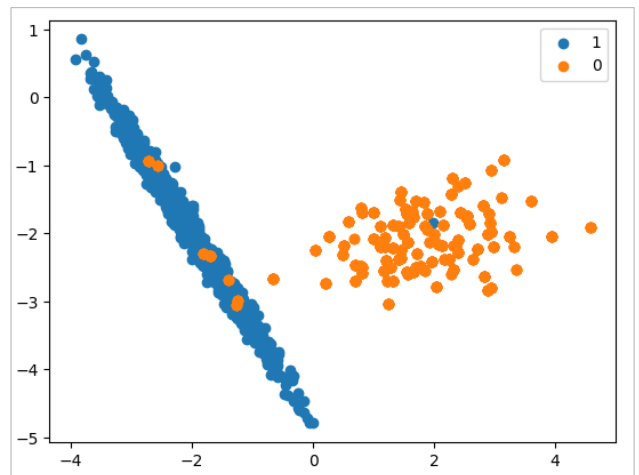


Figure 1: Scatter plot of examples by class label – pre-upsampling (used the same data for all the upsampling methods below, and observe how this data changes after using each technique)

In this code we will see how upsampling is done:

```
df = pd.DataFrame(X, columns=['Column_A', 'Column_B',
'Column_C', 'Column_D'])
df['Y'] = y
over_samples = df[df["Y"] == 1]
under_samples = df[df["Y"] == 0]
print("Original Under Sample Shape", under_samples.shape)
df_res = resample(under_samples,
replace=True,
n_samples=len(over_samples),
random_state=42)
print("Resampled Under Sample Shape", df_res.shape)

X_Res = np.append(df_res[['Column_A', 'Column_B', 'Column_C',
'Column_D']].to_numpy(), over_samples[['Column_A',
'Column_B', 'Column_C', 'Column_D']].to_numpy(), axis=0)
y_res = np.append(df_res['Y'].to_numpy(), over_samples['Y'].
to_numpy())

print('Resampled dataset shape %s' % Counter(y_res))
Original Under Sample Shape (106, 5)
Resampled Under Sample Shape (894, 5)
Resampled dataset shape Counter({0: 894, 1: 894})

# scatter plot of examples by class label
for label, _ in Counter(y_res).items():
    row_ix = where(y_res == label)[0]
    pyplot.scatter(X_Res[row_ix, 0], X_Res[row_ix, 1],
label=str(label))
```